ZentroMart VoiceOps: grounded voice support agent

VoiceOps 2026 capstone · Manav Parihar · Speech: Gnani STT + TTS only · Step 1 model: `openai/gpt-oss-120b` (Groq) · Models that answered the reported Step 2/3 runs (approximate turn counts, logged per answer): gemini-flash-lite-latest (120), qwen/qwen3.8-27b (13), openai/gpt-oss-120b (12), gemini-3.5-flash (2), gemini-3.8-flash (1)

```
customer audio → Gnani STT (en-IN + hi-IN) → agent [hybrid RAG · conversation state · lookup_order · create_ticket · date tool] → Gnani TTS (timbre-v2.5) → spoken reply
```

1. What I built in each step

Step	What it does
1 · LLM only	System prompt that sets the role (ZentroMart's AI support assistant), its limits (no promises, no other customers' data, say when unsure) and an output contract ending with `DECISION:` and `SOURCES:`. No documents, no tools.
2 · Grounded (RAG)	Each `## [SECTION-ID]` becomes one chunk with its doc metadata (doc type, status, effective_from, superseded banner); each catalog row becomes a `products.csv:SKU` chunk (137 chunks). Retrieval = multilingual embeddings (paraphrase-multilingual-MiniLM, local ONNX) in a NumPy vector index + BM25 with a small Hindi/Hinglish/English synonym list, merged by reciprocal-rank fusion and re-ranked by the KB-00.1 precedence (policy > agent guide > help > FAQ > marketing; superseded down-weighted). A product mention also brings in the policy section that defines its catalog code (two-source answers, e.g. `fashion` → RET-2). The model must answer only from the given sections, say "I don't have information" otherwise, and cite the IDs it used. IDs are checked against the real index.
3 · Conversation	Conversation memory (last 10 messages and a structured order state). Order IDs and verification values are detected in code, including spoken digits ("eight nine seven eight…", "aath nau…"), and sent to the dataset's `lookup_order`. One retry, then locked. Order data reaches the LLM only after `verified=True`, and secrets are masked in logs and ticket summaries. `create_ticket` is an LLM tool, and a code check re-prompts the model if it claims a handover without a successful ticket. A `date_calc` tool plus a pre-computed date helper handle business days (Mon–Sat minus holidays) and calendar-day return windows. Decision labels for the steps the code performs (verification, ticket) are set deterministically.
4 · Voice	Gnani REST STT runs each clip in en-IN and hi-IN in parallel. A Hinglish detector tells the agent which language was spoken, and long WAVs are split under 60 s. The agent's reply is rewritten for the ear (dates with month and year, order IDs in groups of four, "49 rupees", no ticket hashes) and spoken with Gnani TTS `timbre-v2.5` (Kaveri for English, Nalini for Hindi). Empty audio gives "Sorry, I didn't catch that". `python run.py --input audio/ --output outputs/` saves the reply text, reply audio, cited chunks, retrieved chunks and tool calls per file. A browser demo (`web/app.py`) records the microphone as WAV.

2. Step 1 vs Step 2 on the 52 practice questions



Metric	Step 1	Step 2
Fully correct (decision ✓, all must_include facts, no forbidden phrase)	**13/52** (25%)	**40/52** (77%)

Right decision (answer / clarify / escalate / refuse)	62%	86%
All required facts present	31%	83%
Gold evidence recall: cited sources / retrieved chunks	– / –	71% / 73%
By language (correct)	en: 9/33, hi: 3/8, hi-en: 1/11	en: 27/33, hi: 6/8, hi-en: 7/11

After the fixes in section 4, re-running only the 12 questions that had failed gave 8/12 correct, which would make Step 2 48/52 if the other answers stay the same.

What I learned. Without documents the model sounds confident but invents ZentroMart-specific numbers: wallet refunds "3–5 days", UPI "2–5 days", a 10-day electronics window (the superseded v3 rule), no student discount instead of "I don't know". It also asks for an order ID instead of answering general policy questions, and it never escalates. Grounding fixed the facts. The gains on date questions came from computing dates in code rather than in the model, and the "I don't know" cases needed an explicit rule not to state what ZentroMart does or does not offer. Scoring is my own automatic check (DATA_DICTIONARY rules with date normalisation), so it is strict about wording: a correct Hindi answer that transliterates "QC" can be marked wrong.

3. Step 3: 23 practice conversations (96 customer turns)

Metric	First run	Final
Turns fully correct (decision, facts, forbidden phrases, expected tool calls)	73/96 (76%)	**89/96** (93%)
Expected tool calls (lookup_order with/without verification, create_ticket)	93%	99%
Decision accuracy	80%	97%
Conversations correct at every turn	5/23	16/23
Forbidden claims ("I have cancelled", "approved", leaked pincode...)	0% of turns	0% of turns

4. What went wrong and how I fixed it

- **Free-tier limits.** Groq allows 8,000 tokens/min, and the first prompt (all agent guides plus 8 verbose chunks) used about 9k per turn. I compacted the chunk rendering and moved KB-00 into retrieval (about 3k tokens per call), and added an automatic fallback chain (Groq gpt-oss-120b → Groq Qwen → Gemini Flash) with per-model cool-downs. Gemini rejected tool histories created by another model (missing `thought_signature`), so the history is replayed as text when that happens.
- **Which model answered.** Because of the free-tier daily caps, most of the reported Step 2/3 turns were answered by fallback models rather than the primary gpt-oss-120b (mix in the header). The numbers are therefore a conservative estimate for the primary model.
- **A weak fallback hurt quality.** In the first Step 2 run (40/52) almost every miss came from Gemini Flash Lite, which skipped the date tool and miscounted business days. I removed it and added a pre-computed date helper for every date the customer mentions.
- **Language drift.** English conversations sometimes got Hinglish replies. The customer's language is now detected in code and stated to the model on every turn.
- **Decision labels and verification turns.** The model escalated on the verification turn when `contacts_last_7_days=2`, or called a verification question "answer". Labels for code-driven steps are now deterministic, and `create_ticket` is not offered on a turn that only answers the verification question.
- **Intro.** STY-1 is guaranteed in code: if the first reply doesn't say "AI", an introduction in the reply's language is prepended.

Still failing in the final runs:

- **PQ0022**: 2026-10-27 / 27 October 2026 / 27 अक्टूबर 2026
- **PQ0023**: ticket / colleague / human / team / connect / टिकट / सहयोगी
- **PQ0027**: 2026-10-15 / 15 October 2026 / 15 अक्टूबर 2026
- **PQ0031**: 30, 10
- **PQ0035**: 2026-10-17 / 17 October 2026 / 17 अक्टूबर 2026
- **PQ0037**: don't have / do not have / not sure / no information / can't find / not able to find / couldn't find / जानकारी नहीं / pata nahi / jaankari nahi / connect / सहयोगी
- **PC002 t4**: expected escalate, got refuse
- **PC004 t4**: expected answer, got answer; missing 2026-11-13 / 13 November 2026 / 13 नवंबर 2026, 49
- **PC008 t3**: expected answer, got escalate
- **PC018 t4**: expected answer, got refuse; missing My Orders / app / free
- **PC019 t4**: expected refuse, got refuse; missing 7
- **PC020 t4**: expected refuse, got refuse; missing 7

What I would fix next: a small LLM judge for semantic scoring of Hindi answers; a cross-encoder re-ranker and Gemini embeddings to raise cited-source precision; streaming STT/TTS (Gnani WebSocket) to cut latency (currently about 15.336 s per turn on free-tier keys); confidence-based "please repeat" using STT disagreement between the en-IN and hi-IN passes.

5. Bonus work

- Dual-language STT (en-IN + hi-IN) with automatic Hindi/Hinglish detection, and Hindi TTS voice switching.
- Spoken-number normalisation for order IDs, SKUs and verification digits in English, Hinglish and Hindi.
- Browser voice demo that records WAV in the page (no ffmpeg needed) and shows decision, tools and cited sections.
- Provider-agnostic LLM layer (any OpenAI-compatible API) with rate-limit-aware fallback, so long batch runs finish on free keys.
- Reproducible evaluation: `python eval/run_eval.py --step 1|2|3 [--asr]` writes every reply, decision, tool call and source to `eval/results/`.